SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	**5**
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	**5**
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	**5**
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	**5**
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	**5**
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	**5**

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	**5**
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	**5**
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	**5**
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	**5**

Code of Conduct:

I Sandra Sudevan(192424422) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Sandrasudevan

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1: Online Shopping Platform – 3NF

Given Relation

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

The online shopping platform stores customer, product, and order information in a single relation. This causes repeated customer and product information.

```
mysql>
mysql> CREATE TABLE Orders(
    ->     OrderID INT PRIMARY KEY,
    ->     CustID INT,
    ->     FOREIGN KEY(CustID) REFERENCES Customer(CustID)
    -> );
Query OK, 0 rows affected (0.962 sec)

mysql>
mysql> CREATE TABLE OrderDetails(
    ->     OrderID INT,
    ->     ProdID INT,
    ->     Qty INT,
    ->     PRIMARY KEY(OrderID, ProdID),
    ->     FOREIGN KEY(OrderID) REFERENCES Orders(OrderID),
    ->     FOREIGN KEY(ProdID) REFERENCES Product(ProdID)
    -> );
Query OK, 0 rows affected (0.908 sec)

mysql>
mysql> SHOW TABLES;
+---------------------+
| Tables_in_shoppingdb |
+---------------------+
| customer            |
| orderdetails        |
| orders              |
| product             |
+---------------------+
4 rows in set (0.544 sec)
```

Step 1: Identify Functional Dependencies

The functional dependencies are:

- OrderID → CustID
  Each order belongs to one customer.
- CustID → CustName
  Each customer ID identifies one customer name.
- ProdID → ProdName, Price
  Each product ID identifies its product name and price.
- (OrderID, ProdID) → Qty
  The quantity depends on both the order and product.

Step 2: Identify Candidate Key

The candidate key is:

(OrderID, ProdID)

because an order can contain multiple products.

Step 3: Identify Dependencies

There are partial dependencies:

- OrderID → CustID
- ProdID → ProdName, Price

There is also a transitive dependency:

- OrderID → CustID → CustName

Step 4: Decompose into 3NF

The relation is divided into:

Customer

Customer(CustID, CustName)

Product

Product(ProdID, ProdName, Price)

Orders

Orders(OrderID, CustID)

OrderDetails

OrderDetails(OrderID, ProdID, Qty)

Advantages

- Customer information is stored only once.
- Product information is stored only once.
- Updating a product price does not require changing many order records.
- Insertion and deletion anomalies are reduced.

Result: The online shopping relation is successfully normalized to **3NF**.

Case Study 2: Hospital Management – BCNF

Given Relation

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

The relation contains patient, doctor, and ward information together.

```
mysql> USE HospitalDB;
Database changed
mysql>
mysql> CREATE TABLE Doctor(
    ->     DocID INT PRIMARY KEY,
    ->     DocName VARCHAR(50),
    ->     Specialization VARCHAR(50)
    -> );
Query OK, 0 rows affected (1.120 sec)

mysql>
mysql> CREATE TABLE Ward(
    ->     WardNo INT PRIMARY KEY,
    ->     WardName VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.283 sec)

mysql>
mysql> CREATE TABLE Patient(
    ->     PID INT PRIMARY KEY,
    ->     PName VARCHAR(50),
    ->     DocID INT,
    ->     WardNo INT,
    ->     FOREIGN KEY(DocID) REFERENCES Doctor(DocID),
    ->     FOREIGN KEY(WardNo) REFERENCES Ward(WardNo)
    -> );
Query OK, 0 rows affected (1.248 sec)

mysql>
mysql> SHOW TABLES;
+---------------------+
| Tables_in_hospitaldb |
+---------------------+
| doctor              |
| patient             |
| ward                |
+---------------------+
3 rows in set (0.277 sec)
```

Step 1: Functional Dependencies

- PID → PName, DocID, WardNo
- DocID → DocName, Specialization
- WardNo → WardName

Step 2: Identify Problems

Insertion Anomaly

If a new doctor joins the hospital but has no patient yet, the doctor information cannot be stored easily.

Update Anomaly

If a doctor's specialization changes, the information may need to be changed in several patient records.

Deletion Anomaly

If the last patient assigned to a doctor is deleted, the doctor's information may also be lost.

Step 3: BCNF Decomposition

Patient

Patient(PID, PName, DocID, WardNo)

Doctor

Doctor(DocID, DocName, Specialization)

Ward

Ward(WardNo, WardName)

Explanation

In the new relations:

- PID determines all attributes in Patient.
- DocID determines all attributes in Doctor.
- WardNo determines all attributes in Ward.

Therefore, every determinant is a candidate key in its relation.

Result: The hospital database is normalized to **BCNF**, eliminating major redundancy and anomalies.

Case Study 3: University Enrollment – 2NF

Given Relation

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

This relation contains student and course information along with enrollment grades.

```
mysql> CREATE DATABASE UniversityDB;
ERROR 1007 (HY000): Can't create database 'universitydb'; database exists
mysql> USE UniversityDB;
Database changed
mysql>
mysql> CREATE TABLE Student(
    ->     SID INT PRIMARY KEY,
    ->     SName VARCHAR(50)
    -> );
ERROR 1050 (42S01): Table 'student' already exists
mysql>
mysql> CREATE TABLE Course(
    ->     CourseID INT PRIMARY KEY,
    ->     CourseName VARCHAR(50),
    ->     Faculty VARCHAR(50)
    -> );
ERROR 1050 (42S01): Table 'course' already exists
mysql>
mysql> CREATE TABLE Enrollment(
    ->     SID INT,
    ->     CourseID INT,
    ->     Grade VARCHAR(5),
    ->     PRIMARY KEY(SID, CourseID),
    ->     FOREIGN KEY(SID) REFERENCES Student(SID),
    ->     FOREIGN KEY(CourseID) REFERENCES Course(CourseID)
    -> );
ERROR 1050 (42S01): Table 'enrollment' already exists
mysql>
mysql> SHOW TABLES;
+------------------------+
| Tables_in_universitydb |
+------------------------+
| course                 |
| enrollment             |
| faculty                |
| student                |
+------------------------+
4 rows in set (0.035 sec)
```

Step 1: Functional Dependencies

- SID → SName
- CourseID → CourseName, Faculty
- (SID, CourseID) → Grade

Step 2: Candidate Key

The candidate key is:

(SID, CourseID)

because a student can enroll in multiple courses.

Step 3: Identify Partial Dependencies

SName depends only on SID, not on the complete key.

CourseName and Faculty depend only on CourseID, not on the complete key.

Therefore, partial dependencies exist.

Step 4: Decompose into 2NF

Student

Student(SID, SName)

Course

Course(CourseID, CourseName, Faculty)

Enrollment

Enrollment(SID, CourseID, Grade)

Explanation

Now:

- Student name depends only on Student ID.
- Course details depend only on Course ID.
- Grade depends on the complete combination of Student ID and Course ID.

Thus, partial dependencies are removed.

Result: The University Enrollment relation is successfully normalized to **2NF**.

Case Study 4: Airline Booking – BCNF

Given Relation

Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

The relation contains passenger, flight, and booking information.

```
mysql> USE AirlineDB;
Database changed
mysql>
mysql> CREATE TABLE Passenger(
    ->     PassID INT PRIMARY KEY,
    ->     PassName VARCHAR(50)
    -> );
Query OK, 0 rows affected (1.775 sec)

mysql>
mysql> CREATE TABLE Flight(
    ->     FlightNo VARCHAR(10) PRIMARY KEY,
    ->     DeptCity VARCHAR(50),
    ->     ArrCity VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.446 sec)

mysql>
mysql> CREATE TABLE Booking(
    ->     FlightNo VARCHAR(10),
    ->     PassID INT,
    ->     Date DATE,
    ->     Seat VARCHAR(10),
    ->     PRIMARY KEY(FlightNo, Date, PassID),
    ->     FOREIGN KEY(FlightNo) REFERENCES Flight(FlightNo),
    ->     FOREIGN KEY(PassID) REFERENCES Passenger(PassID)
    -> );
Query OK, 0 rows affected (3.033 sec)

mysql>
mysql> SHOW TABLES;
+---------------------+
| Tables_in_airlinedb |
+---------------------+
| booking             |
| flight              |
| passenger           |
+---------------------+
3 rows in set (0.481 sec)
```

Step 1: Functional Dependencies

- PassID → PassName
- FlightNo → DeptCity, ArrCity
- (FlightNo, Date, PassID) → Seat

Step 2: Candidate Key

The candidate key is:

(FlightNo, Date, PassID)

This combination uniquely identifies a booking.

Step 3: Identify Dependencies

Passenger information depends only on PassID.

Flight information depends only on FlightNo.

Seat information depends on the complete booking.

Step 4: BCNF Decomposition

Passenger

Passenger(PassID, PassName)

Flight

Flight(FlightNo, DeptCity, ArrCity)

Booking

Booking(FlightNo, Date, PassID, Seat)

Explanation

The decomposition separates passenger details, flight details, and booking details.

- PassID is the key of Passenger.
- FlightNo is the key of Flight.
- (FlightNo, Date, PassID) is the key of Booking.

Therefore, each determinant is a candidate key in its respective relation.

Result: The airline booking relation is normalized to **BCNF**.

Case Study 5: HR System – 3NF

Given Relation

Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName)

The relation stores employee, department, and manager information together.

```
mysql> USE HRDB;
Database changed
mysql>
mysql> CREATE TABLE Manager(
    ->     ManagerID INT PRIMARY KEY,
    ->     ManagerName VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.746 sec)

mysql>
mysql> CREATE TABLE Department(
    ->     DeptID INT PRIMARY KEY,
    ->     DeptName VARCHAR(50),
    ->     ManagerID INT,
    ->     FOREIGN KEY(ManagerID) REFERENCES Manager(ManagerID)
    -> );
Query OK, 0 rows affected (1.773 sec)

mysql>
mysql> CREATE TABLE Employee(
    ->     EmpID INT PRIMARY KEY,
    ->     EmpName VARCHAR(50),
    ->     DeptID INT,
    ->     ManagerID INT,
    ->     FOREIGN KEY(DeptID) REFERENCES Department(DeptID),
    ->     FOREIGN KEY(ManagerID) REFERENCES Manager(ManagerID)
    -> );
Query OK, 0 rows affected (1.287 sec)

mysql>
mysql> SHOW TABLES;
+----------------+
| Tables_in_hrdb |
+----------------+
| department     |
| employee       |
| manager        |
+----------------+
3 rows in set (0.411 sec)
```

Step 1: Functional Dependencies

- EmpID → EmpName, DeptID, ManagerID
- DeptID → DeptName, ManagerID
- ManagerID → ManagerName

Step 2: Identify Transitive Dependencies

There are two important transitive dependencies:

EmpID → DeptID → DeptName

and

EmpID → ManagerID → ManagerName

This means department and manager details do not directly depend on the employee.

Step 3: Decompose into 3NF

Employee

Employee(EmpID, EmpName, DeptID, ManagerID)

Department

Department(DeptID, DeptName, ManagerID)

Manager

Manager(ManagerID, ManagerName)

Explanation

Now:

- Employee details are stored in Employee.
- Department details are stored in Department.
- Manager details are stored in Manager.

This prevents the same department and manager information from being repeated for every employee.

Advantages

- Reduces data redundancy.
- Prevents update anomalies.
- Prevents insertion anomalies.
- Prevents deletion anomalies.
- Makes the database easier to maintain.

Result: The HR database is successfully normalized to **3NF** by removing transitive dependencies.

6. Library System – 4NF

Given Relation

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Multivalued Dependencies (MVDs)

A member can borrow multiple books, and a book can have multiple authors/genres independently.

```
mysql>
mysql> CREATE TABLE Borrow(
    ->     MemberID INT,
    ->     BookID VARCHAR(10),
    ->     BorrowDate DATE,
    ->     PRIMARY KEY(MemberID, BookID),
    ->     FOREIGN KEY(MemberID) REFERENCES Member(MemberID),
    ->     FOREIGN KEY(BookID) REFERENCES Book(BookID)
    -> );
Query OK, 0 rows affected (3.442 sec)

mysql>
mysql> CREATE TABLE BookAuthor(
    ->     BookID VARCHAR(10),
    ->     Author VARCHAR(50),
    ->     PRIMARY KEY(BookID, Author),
    ->     FOREIGN KEY(BookID) REFERENCES Book(BookID)
    -> );
Query OK, 0 rows affected (0.350 sec)

mysql>
mysql> CREATE TABLE BookGenre(
    ->     BookID VARCHAR(10),
    ->     Genre VARCHAR(50),
    ->     PRIMARY KEY(BookID, Genre),
    ->     FOREIGN KEY(BookID) REFERENCES Book(BookID)
    -> );
Query OK, 0 rows affected (0.360 sec)

mysql>
mysql> SHOW TABLES;
+--------------------+
| Tables_in_librarydb |
+--------------------+
| book               |
| bookauthor         |
| bookgenre          |
| borrow             |
| member             |
+--------------------+
```

The important MVDs are:

- MemberID →→ BookID
- BookID →→ Author
- BookID →→ Genre

Problem

The relation may contain unnecessary combinations of books, authors, and genres. This creates redundancy and insertion, update, and deletion anomalies.

4NF Decomposition

Member

Member(MemberID, MemberName)

Borrow

Borrow(MemberID, BookID, BorrowDate)

Book

Book(BookID, Title)

BookAuthor

BookAuthor(BookID, Author)

BookGenre

BookGenre(BookID, Genre)

Explanation

The independent multivalued facts are separated into different relations. Therefore, the MVDs are removed and the relations satisfy **4NF**.

Result: The Library database is successfully decomposed into **4NF**.

7. Telecom Billing Schema – Candidate Keys and Functional Dependencies

Example Relation

Billing(CustomerID, CustomerName, PhoneNo, PlanID, PlanName, CallID, CallDate, Duration, Amount)

```
mysql> USE TelecomDB;
Database changed
mysql>
mysql> CREATE TABLE Customer(
    ->     CustomerID INT PRIMARY KEY,
    ->     CustomerName VARCHAR(50),
    ->     PhoneNo VARCHAR(15)
    -> );
Query OK, 0 rows affected (0.485 sec)

mysql>
mysql> CREATE TABLE Plan(
    ->     PlanID VARCHAR(10) PRIMARY KEY,
    ->     PlanName VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.310 sec)

mysql>
mysql> CREATE TABLE Billing(
    ->     CallID VARCHAR(10) PRIMARY KEY,
    ->     CustomerID INT,
    ->     PlanID VARCHAR(10),
    ->     CallDate DATE,
    ->     Duration INT,
    ->     Amount DECIMAL(10,2),
    ->     FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID),
    ->     FOREIGN KEY(PlanID) REFERENCES Plan(PlanID)
    -> );
Query OK, 0 rows affected (0.923 sec)

mysql>
mysql> SHOW TABLES;
+--------------------+
| Tables_in_telecomdb |
+--------------------+
| billing            |
| customer           |
| plan               |
+--------------------+
```

Functional Dependencies

- CustomerID → CustomerName, PhoneNo
- PhoneNo → CustomerID
- PlanID → PlanName
- CallID → CustomerID, CallDate, Duration, Amount
- (CustomerID, CallID) → CallDate, Duration, Amount

Candidate Keys

Possible candidate keys are:

- CustomerID for customer information.
- PhoneNo can also identify a customer if phone numbers are unique.

- CallID uniquely identifies a call record.

Primary Key

For the billing/call transaction table:

CallID can be selected as the primary key.

Explanation

Candidate keys are attributes or combinations of attributes that uniquely identify a tuple. A primary key is one selected candidate key used to uniquely identify records.

Result: Candidate keys, primary key, and functional dependencies are identified before normalization.

8. Retail Inventory System – Complete Normalization

Given Relation

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

```
mysql> USE RetailDB;
Database changed
mysql>
mysql> CREATE TABLE Supplier(
    ->     SupplierID VARCHAR(10) PRIMARY KEY,
    ->     SupplierName VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.588 sec)

mysql>
mysql> CREATE TABLE Product(
    ->     PID INT PRIMARY KEY,
    ->     PName VARCHAR(50),
    ->     SupplierID VARCHAR(10),
    ->     Category VARCHAR(50),
    ->     Price DECIMAL(10,2),
    ->     Warehouse VARCHAR(50),
    ->     FOREIGN KEY(SupplierID) REFERENCES Supplier(SupplierID)
    -> );
Query OK, 0 rows affected (0.945 sec)

mysql>
mysql> SHOW TABLES;
+--------------------+
| Tables_in_retaildb |
+--------------------+
| product            |
| supplier           |
+--------------------+
2 rows in set (0.086 sec)

mysql>
```

Functional Dependencies

- PID → PName, SupplierID, Category, Price, Warehouse
- SupplierID → SupplierName

There is a transitive dependency:

PID → SupplierID → SupplierName

1NF

All attributes contain atomic values.

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

2NF

Since PID is a single-attribute key, there are no partial dependencies.

Therefore, the relation is already in **2NF**.

3NF

Remove the transitive dependency by decomposing:

Product

Product(PID, PName, SupplierID, Category, Price, Warehouse)

Supplier

Supplier(SupplierID, SupplierName)

BCNF

In both relations, the determinant is a candidate key:

- PID → product details
- SupplierID → supplier name

Therefore, the relations satisfy **BCNF**.

Result: The retail inventory system is completely normalized up to **BCNF**.

9. School ERP – Normalization up to BCNF

Poorly Normalized Relation

School(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Grade)

```
mysql>
mysql> CREATE TABLE Student(
    ->     StudentID INT PRIMARY KEY,
    ->     StudentName VARCHAR(50),
    ->     ClassID VARCHAR(10),
    ->     FOREIGN KEY(ClassID) REFERENCES Class(ClassID)
    -> );
Query OK, 0 rows affected (0.664 sec)

mysql>
mysql> CREATE TABLE Subject(
    ->     SubjectID VARCHAR(10) PRIMARY KEY,
    ->     SubjectName VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.294 sec)

mysql>
mysql> CREATE TABLE Enrollment(
    ->     StudentID INT,
    ->     SubjectID VARCHAR(10),
    ->     Grade VARCHAR(5),
    ->     PRIMARY KEY(StudentID, SubjectID),
    ->     FOREIGN KEY(StudentID) REFERENCES Student(StudentID),
    ->     FOREIGN KEY(SubjectID) REFERENCES Subject(SubjectID)
    -> );
Query OK, 0 rows affected (0.943 sec)

mysql>
mysql> SHOW TABLES;
+--------------------+
| Tables_in_schooldb |
+--------------------+
| class              |
| enrollment         |
| student            |
| subject            |
| teacher            |
+--------------------+
5 rows in set (0.334 sec)
```

Functional Dependencies

- StudentID → StudentName, ClassID
- ClassID → ClassName, TeacherID
- TeacherID → TeacherName
- SubjectID → SubjectName
- (StudentID, SubjectID) → Grade

Problems

The original table has:

- Repeated student information.
- Repeated class information.

- Repeated teacher information.
- Repeated subject information.
- Update anomalies.
- Insertion anomalies.
- Deletion anomalies.
- Transitive dependencies.

BCNF Decomposition

Student

Student(StudentID, StudentName, ClassID)

Class

Class(ClassID, ClassName, TeacherID)

Teacher

Teacher(TeacherID, TeacherName)

Subject

Subject(SubjectID, SubjectName)

Enrollment

Enrollment(StudentID, SubjectID, Grade)

Justification

In each relation, the determinant is a candidate key:

- StudentID → Student details
- ClassID → Class details
- TeacherID → Teacher details
- SubjectID → Subject details
- (StudentID, SubjectID) → Grade

Therefore, all decomposed relations satisfy **BCNF**.

Result: The School ERP schema is fully normalized up to **BCNF**.

10. Movie Streaming Platform – 5NF

Example Relation

Streaming(MovieID, ActorID, DirectorID, PlatformID)

Assume a movie can have multiple actors, directors, and streaming platforms independently.

```
mysql> USE MovieDB;
Database changed
mysql>
mysql> CREATE TABLE MovieActor(
    ->     MovieID VARCHAR(10),
    ->     ActorID VARCHAR(10),
    ->     PRIMARY KEY(MovieID, ActorID)
    -> );
Query OK, 0 rows affected (1.098 sec)

mysql>
mysql> CREATE TABLE MovieDirector(
    ->     MovieID VARCHAR(10),
    ->     DirectorID VARCHAR(10),
    ->     PRIMARY KEY(MovieID, DirectorID)
    -> );
Query OK, 0 rows affected (0.320 sec)

mysql>
mysql> CREATE TABLE MoviePlatform(
    ->     MovieID VARCHAR(10),
    ->     PlatformID VARCHAR(10),
    ->     PRIMARY KEY(MovieID, PlatformID)
    -> );
Query OK, 0 rows affected (0.336 sec)

mysql>
mysql> SHOW TABLES;
+-------------------+
| Tables_in_moviedb |
+-------------------+
| movieactor        |
| moviedirector     |
| movieplatform     |
+-------------------+
3 rows in set (0.275 sec)
```

Join Dependencies

The relation may contain the following independent relationships:

- MovieID ↔ ActorID
- MovieID ↔ DirectorID
- MovieID ↔ PlatformID

This can create unnecessary combinations of actors, directors, and platforms.

5NF Decomposition

MovieActor

MovieActor(MovieID, ActorID)

MovieDirector

MovieDirector(MovieID, DirectorID)

MoviePlatform

MoviePlatform(MovieID, PlatformID)

Lossless-Join Verification

The original relation can be reconstructed using natural joins:

MovieActor ⋈ MovieDirector ⋈ MoviePlatform

If the original valid combinations are obtained without adding spurious tuples, the decomposition is **lossless**.

Explanation

5NF removes redundancy caused by complex join dependencies. Each independent many-to-many relationship is stored separately.

Result: The movie streaming schema is decomposed into **5NF** relations with a lossless join.